

Operatorerna == och =

En praktisk operator som ännu inte överlagrats är jämförelseoperatoren ==. Vill vi med vår nuvarande definition av klassen `Vector` jämföra två vektorer måste vi manuellt jämföra varje medlem, vilket är fult och arbetsdrygt. Istället överlagrar vi operatoren == och implementerar den nödvändiga funktionaliteten:

```
// prototyp
bool operator==(const Vector & V) const;

// implementation
bool Vector::operator==(const Vector & V) const {
    // jämför V och denna vektor
    if ( m_X == V.m_X && m_Y == V.m_Y && m_Z == V.m_Z ) {
        // vektorerna är lika
        return true;
    }

    // de är olika
    return false;
}
```

Ett program som använder sig av == kan se ut på följande sätt:

```
#include <iostream>
#include "Vector8.h"

int main () {
    Vector V1 ( 1, 2, 3 );
    Vector V2 ( 6, 5, 4 );
    Vector V3 ( 1, 2, 3 );

    // jämför V1 och V2
    if ( V1 == V2 )
        cout << "V1 == V2" << endl;
    else
        cout << "V1 != V2" << endl;

    // jämför V1 och V3
    if ( V1 == V3 )
        cout << "V1 == V3" << endl;
    else
        cout << "V1 != V3" << endl;
}
```

Kör man programmet får man följande utskrift:

```
% ./TestVector8
V1 != V2
V1 == V3
%
```

Enda stora skillnaden mellan implementationen av denna överlagrade operator och de tidigare överlagrade är returtypen som är `bool`. Men vid närmare eftertanke är det ju den enda möjliga

returntypen för en jämförelse som ju kan antingen vara sann eller falsk. Vi kan på samma gång överlagra även != för att göra det lätt att jämföra vektorer med varandra:

```
// prototyp
bool operator!= (const Vector & V) const;

// implementation
bool Vector::operator!= (const Vector & V) const {
    // använd negerad jämförelse
    return ! ( *this == V );
}
```

Koden ovan kan behöva några förklarande ord. Vi använder oss av den tidigare överlagrade operatör == så att vi inte behöver skriva en ny och lång if-sats, men negerar det resultat som vi får. Så om de två vektorerna är lika varandra så är det *inte* olika varandra. Vi utför alltså först en jämförelse och sedan negerar resultatet. Här får vi nytta av att kunna referera till nuvarande objekt via pekaren `this`. Eftersom det inte finns någon överlagrad operatör för att hantera pekare till vektorer måste vi först avreferera `this`, och först sedan utföra jämförelsen med `V`. Parentesen runt jämförelsen är nödvändig, eftersom `!` har högre precedens än `==`, och följdaktligen försöker kompilatorn först negera `*this` och sedan utföra jämförelsen. Vi vill dock göra det hela i motsatt ordning, därav parentesen.

Tilldelningsoperatör =

En operatör som vi egentligen inte behöver överlagra för vår enkla klass är operatör `=`, d.v.s. normal tilldelning. Kompilatorn skapar alltid automatiskt en version av tilldelning ifall programmeraren inte gör det, och för klassen `Vector` skulle den versionen duga mycket väl. Det som den kompilatorgenererade versionen gör är att den skapar kod för att kopiera alla medlemmar från originalobjektet till det kopierade objektet. En dylik approach fungerar fint för vår klass, men kanske inte för en mer avancerad klass som innehåller pekare och/eller allokerat minne. Se t.ex. [avsnittet Mera om copy-konstruktor i Kapitel 14](#) för ett exempel på en klass som inte kan använda den normala tilldelningen utan att orsaka problem. Notera att klassen där även heter `Vector`, men där avses en container, inte en matematisk vektor.

Vi skall dock explicit överlagra operatör `=` i demonstrationssyfte. För andra mer komplicerade klasser är definitionen i princip samma, men den egentliga koden inne i metoden skiljer sig förstås. Vi får följande metod:

```
// prototyp
Vector & operator= (const Vector & V);

// implementation
Vector & Vector::operator= (const Vector & V) {
    // tilldelas vi värdet av oss själva?
    if ( this == &V ) {
        // jep, gör inget i så fall
        return *this;
    }

    // utför tilldelning
    m_X = V.m_X;
    m_Y = V.m_Y;
    m_Z = V.m_Z;

    // returnera oss själva
    return *this;
}
```

Denna metod kräver även lite förklaring. För det första är returvärdet i denna metod en *referens* till en `Vector`. Alla andra metoder har returnerat en "vanlig" `Vector` och inte en referens. Skillnaden ligger i att de hade i så fall returnerar en referens till en lokal variabel, som ju skall förstöras direkt då metoden är avslutad. En referens till ett förstört objekt är inte giltig, därför returneras en kopia. Men vid överlagring av operatoren `=` returneras inte en referens till en lokal variabel, utan till själva klassen för vilken metoden anropats, d.v.s. `this`. Vi avrefererar `this` för att returnera en `Vector` och inte en `Vector *`. Tilldelningen sker till aktuellt objekt, precis som med operatoren `+=`.

De första raderna i metoden kontrollerar att vi inte försöker tilldela objektet till sig själv, d.v.s. göra något i stil med:

```
Vector v1 ( 1, 2, 3 );

// tilldela till sig själv
v1 = v1;
```

Om så görs skall inget speciellt göras. Man kan kontrollera om adressen för `v` är samma som `this`. Om så är fallet är det samma objekt, eftersom de finns på samma adress i minnet.

Den skarpsynte funderar nu säkert varför måste vi returnera en referens till en `Vector` då vi överlagrar tilldelning, räcker det inte med att enbart kopiera data från ursprungsvektorn? Orsaken till detta är att man i C++ (och C) kan skriva multipla tilldelningar på samma rad, t.ex. är följande giltigt:

```
Vector v1, v2, v3;

// multipel tilldelning
v1 = v2 = v3 = Vector (1, 1, 1);
```

Här vill vi att `v1`, `v2` och `v3` skall alla få värdet av den sista vektorn. Tilldelningarna utförs från höger till vänster, så `v1` tilldelas sist. Normalt brukar man försöka undvika att använda multipel tilldelning, men eftersom det är laglig C++ måste även en överlagrad operator `=` understöda syntaxen. Om vi således inte returnerar en `Vector` i vår metod så "stoppas" sekvensen av tilldelningar.

Vi har nu definierat de flesta operatorer som är vettiga för en vektor. Man kan även lägga till annan funktionalitet såsom t.ex. rotationer, kryssprodukter o.s.v., men de lämnas som en övning till läsaren :-)

[Förutgående](#)
Överlagring och friends

[Hem](#)
[Upp](#)

[Nästa](#)
Överlagring av << och >>